

İsimler, Bağlamalar ve Kapsam

Names, Bindings, and Scopes

Kapsam: 5.1 Giriş · 5.2 İsimler · 5.3 Değişkenler

Kaynak: Sebesta — Concepts of Programming Languages, 12. Baskı

5.1 Giriş (Introduction)

Zorunlu (imperative) programlama dilleri, altta yatan **von Neumann bilgisayar mimarisinin** soyutlamalarıdır. Bu mimari iki temel bileşenden oluşur:

- ◆ **Bellek (Memory):** Hem komutları hem de verileri depolar.
- ◆ **İşlemci (Processor):** Belleğin içeriğini değiştiren işlemler sağlar.

Programlama dillerindeki bellek hücresi soyutlamaları **değişkenlerdir (variables)**. Bazı soyutlamalar donanım düzeyine çok yakındır (örneğin bir tam sayı değişkeni doğrudan birkaç bellekte depolanır); bazıları ise çok uzaktır (örneğin üç boyutlu bir dizi, yazılım eşleme fonksiyonu gerektirir).

İ Temel Fikir

Değişkenin en temel özelliği tiptir (type). Tip tasarımı kapsamlı kararlar gerektiren karmaşık bir konudur (Bölüm 6). Bu kararların en kritikleri değişkenlerin **kapsam (scope)** ve **ömür (lifetime)** özellikleridir.

5.1.1 Fonksiyonel Dillerde Değişkenler

Fonksiyonel dillerde ifadelere isim verilebilir. Bu isimli ifadeler, zorunlu dillerdeki atama ifadelerine benzese de *temel olarak farklıdır*: bir kez bağlandıktan sonra değiştirilemezler. Bu yüzden zorunlu dillerdeki **isimli sabitlere (named constants)** daha yakındırlar. Saf fonksiyonel dillerde zorunlu dillerdeki değişken kavramı yoktur; ancak pek çok fonksiyonel dil her iki türü de destekler.

Kitap Notasyonu

Kitaptaki Dil Ailesi Notasyonu: "Fortran" ifadesi Fortran'ın tüm sürümleri anlamına gelir. "C" ifadesi orijinal C, C89 ve C99'u kapsar. "C-tabanlı diller" ise C, Objective-C, C++, Java ve C# demektir. "Fortran 95+" ifadesi Fortran 95 ile başlayan tüm sürümler anlamına gelir.

5.2 İsimler (Names)

Değişkenlerin temel özelliklerinden biri olan **isim (name)**, değişken tartışmasına girmeden önce ele alınmalıdır. İsimler; alt programlar, biçimsel parametreler ve diğer program yapılarıyla da ilişkilidir. **Tanımlayıcı (identifier)** terimi çoğunlukla isimle birbirinin yerine kullanılır.

5.2.1 Tasarım Soruları (Design Issues)

İki Temel Tasarım Sorusu

- İsimler büyük/küçük harfe duyarlı mı? (*case sensitive?*)
- Dilin özel sözcükleri **ayrılmış sözcük (reserved word)** mu yoksa **anahtar kelime (keyword)** mi?

5.2.2 İsim Biçimleri (Name Forms)

İsim, programdaki bir varlığı tanımlamak için kullanılan **karakter dizisidir**. Çoğu programlama dilinde isimler aynı yapıyı izler: **bir harfle başlar**, ardından harf, rakam veya alt çizgi (_) karakterleri gelir.

Dil	Uzunluk Kuralı	Özel Karakter Kuralı
C99	Dahili: 63 karakter; Harici (linker): 31 karakter	-
Java / C#	Sınır yok, tüm karakterler anlamlı	-
C++	Standart sınır tanımlamaz; derleyici kısıtlayabilir	-
PHP	Sınır yok	Her değişken \$ ile başlamalı
Perl	Sınır yok	\$ → skalar, @ → dizi, % → hash
Ruby	Sınır yok	@ → instance değişkeni, @@ → class değişkeni

5.2.2.1 camelCase ve alt_çizgi Stili

1970-80'lerde alt çizgi kullanımı yaygınken, günümüzde C-tabanlı dillerde **camelCase (deve-büyük-harf) notasyonu** tercih edilmektedir: çok kelimeli bir isimde ilk kelime hariç her kelimenin ilk harfi büyük yazılır.

```
// camelCase (C-tabanlı dillerde yaygın)
int myStackSize;
String firstName;
```

```
// alt_çizgi stili (eski C, Python)
int my_stack_size;
string first_name;
```

İsim stillerinin karşılaştırması — stil tercihi dil tasarım kararı değildir

5.2.2.2 Büyük/Küçük Harf Duyarlılığı (Case Sensitivity)

C, C++, Java ve C# gibi dillerde büyük/küçük harf **farklı isimler** oluşturur. Aşağıdaki üç isim C++'da tamamen ayrı varlıklardır:

```
// C++'da üç farklı değişken:
int rose;    // küçük harf
int ROSE;    // büyük harf
int Rose;    // karma harf

// Java'da zorunlu büyük/küçük: parseInt (doğru)
int x = Integer.parseInt("42");    // ✓
int y = Integer.ParseInt("42");    // x Derleme hatası!
```

Case sensitivity örneği

Avantajları	Dezavantajları
Aynı isim, farklı kapsamlarda anlam farklılaştırılabilir	Okunabilirliği azaltır: Rose ve rose benzer görünür ama farklıdır
Geliştiriciye esneklik sağlar	Yazılabilirliği zorlaştırır: parseInt mi? ParseInt mi?
C'de kural: değişken adlarında büyük harf yok → sorun yok	Java/C# predefined isimler hem büyük hem küçük içerir → kaçınılmaz

5.2.3 Özel Sözcükler (Special Words)

Programlama dillerindeki özel sözcükler iki işlev görür: (1) gerçekleştirilecek eylemleri isimlendirerek programı daha okunabilir kılar, (2) deyimlerin sözdizimsel bölümlerini birbirinden ayırır.

Ayrılmış Sözcük (Reserved Word)	Anahtar Kelime (Keyword)
Programcı tarafından yeniden tanımlanamaz	Belirli bağlamda özel anlam taşır; yeniden tanımlanabilir
Çoğu modern dil: Java, C, C++, Python, C#...	Fortran bu yaklaşımı kullanır
Sorun: Çok fazla reserved word varsa kullanıcı anlamlı isim bulmakta zorlanır	Sorun: Yeniden tanımlama karmaşıklık ve hatalara yol açabilir
Aşırı örnek: COBOL'da 300 reserved word var! LENGTH, BOTTOM, DESTINATION, COUNT gibi yaygın isimler listede	Fortran'da INTEGER REAL geçerli bir atama ifadesidir (INTEGER değişkenine REAL değeri atama)

i Predefined İsimler

Çoğu dilde, Java paketleri veya C/C++ kütüphanelerinde tanımlanan isimler **ön tanımlı (predefined)**'dir. Bu isimler, programda ancak **açıkça import edildikten sonra** görünür hale gelir. Import edildikten sonra ise artık yeniden tanımlanamaz.

📅 Tarih Notu

Tarihsel Not: En erken programlama dilleri tek karakterli isimler kullandı — bu, erken programlamanın büyük ölçüde matematiksel olduğunu yansıtır (matematikçiler bilinmeyenleri tek harfle gösterir). **Fortran I**, gelenekten koparak isimlerde 6 karaktere kadar izin verdi; bu büyük bir devrimdi.

5.3 Değişkenler (Variables)

Bir program değişkeni, bilgisayar belleğinin bir veya daha fazla hücresinin **soyutlamasıdır**. Programcılar genellikle değişkenleri bellek konumları için isimler olarak düşünür; ancak bir değişken yalnızca bir isimden çok daha fazlasını kapsar.

Makine dilinden assembly diline geçiş, mutlak sayısal bellek adreslerinin isimlerle değiştirilmesiydi; bu, programları çok daha okunabilir, yazılabilir ve bakım yapılabilir hale getirdi. Çevirici (assembler), isimleri gerçek adreslere dönüştürürken adresleri de kendisi seçiyordu.

i Değişken Sextuple'ı

Bir değişken altı özellikten oluşan bir altılı (sextuple) ile karakterize edilir: (isim, adres, değer, tip, ömür, kapsam)

#	Özellik (Attribute)	İngilizce	Kısa Açıklama
1	İsim	Name	Değişkeni tanımlayan sembolik etiket
2	Adres	Address	Bellek konumu — l-value olarak da bilinir
3	Değer	Value	Bellek hücresinin içeriği — r-value
4	Tip	Type	Değer aralığı ve tanımlı işlemler kümesi
5	Ömür	Lifetime	Belirli bir bellek konumuna bağlı kalınan süre
6	Kapsam	Scope	Değişkenin görünür olduğu deyimler aralığı

5.3.1 İsim (Name)

Değişken isimleri programlardaki en yaygın isimlerdir. İsim tasarımı Section 5.2'de ayrıntılı biçimde ele alınmıştır. Çoğu değişkenin bir ismi vardır; ancak ismi olmayanlar da vardır — bunlar **açık heap-dinamik değişkenlerdir (explicit heap-dynamic variables)** ve Section 5.4.3.3'te incelenir.

5.3.2 Adres (Address)

Bir değişkenin **adresi**, ilişkilendirildiği makine bellek adresidir. Bu ilişki görüldüğü kadar basit değildir:

- ◆ Aynı değişken, program yürütümünün farklı noktalarında **farklı adreslerle** ilişkilendirilebilir.
- ◆ Örneğin, bir alt programın yerel değişkeni çalışma zamanı yığınının (run-time stack) ayrılmışsa, farklı çağrılar farklı adreslere yol açabilir.
- ◆ Bu, aynı değişkenin farklı **örnekleri (instantiations)** anlamına gelir.

i l-value ve r-value

- ◆ **l-value (sol-değer):** Değişkenin adresidir. Bir değişken adının atama ifadesinin **sol** tarafında görüldüğünde gereken şeydir.
- ◆ **r-value (sağ-değer):** Değişkenin değeridir. Bir değişken adının atama ifadesinin **sağ** tarafında görüldüğünde gereken şeydir.

```
// x = y + 1 ifadesinde:
// x -> l-value kullanılır (adres gerekli, deger yazilacak)
// y -> r-value kullanılır (adres bulunur, icerik okunur)
int x, y = 5;
x = y + 1; // x'in l-value = adresi; y'nin r-value = 5
```

l-value ve r-value kavramlari

5.3.2.1 Takma Adlar (Aliases)

Aynı bellek konumuna erişmek için birden fazla değişken adı kullanılabiliyorsa, bu değişkenlere **alias (takma ad)** denir.

⚠ Aliasing Tehlikesi

Aliasing, bir değişkenin değerinin **farklı bir değişken üzerinden** değiştirilebileceği anlamına gelir. Örneğin total ve sum alias iseler, total'a yapılan her atama sum'u da etkiler. Bu:

- ◆ Okunabilirliği azaltır — okuyucu iki farklı ismin aynı hücre olduğunu her an akılda tutmalı
- ◆ Program doğrulamasını zorlaştırır
- ◆ Bakımı güçleştirir

Alias Kaynağı	Açıklama	Örnek Dil
Union tipleri	Union içindeki farklı alanlar aynı bellek bölgesini paylaşır	C, C++
İki pointer aynı konumu gösteriyor	Pointer'lar dereference edildiğinde aynı bellek hücresine erişilir	C, C++, Java
Referans değişkenler	İki referans aynı nesneyi gösterirse alias oluşur	C++, Java, C#
C++ pointer → isimli değişken	Pointer dereference edilince değişken adıyla alias yapar	C++
Alt program parametreleri	Parametre geçirme mekanizmaları alias yaratabilir (Bölüm 9)	Birçok dil

```
// C++'da alias örnekleri:
```

```
// 1) İki pointer alias:
```

```
int x = 10;
int *p1 = &x;
int *p2 = &x; // p1, p2 ve x üçü de alias
*p1 = 42;      // x de 42 oldu!
printf("%d", *p2); // 42 – alias etkisi
```

```
// 2) Union alias:
```

```
union { int i; float f; } u;
u.i = 65;
// u.f ile erişildiğinde aynı bellek farklı yorumlanır
```

Alias örnekleri — C++

5.3.3 Tip (Type)

Bir değişkenin **tipi**, iki şeyi belirler:

- ◆ **Değer aralığı (range of values):** Değişkenin alabileceği değerler kümesi
- ◆ **İşlemler kümesi (set of operations):** O değerler üzerinde tanımlı işlemler

```
// Java'da int tipi:  
// Değer aralığı: -2,147,483,648 ile +2,147,483,647  
// Tanımlı işlemler: + - * / % (ve karşılaştırma, bit işlemleri...)  
  
int a = 2_000_000_000; // geçerli  
int b = 3_000_000_000; // HATA! aralık dışı  
  
// double tipi farklı aralık ve farklı işlemler sunar  
double pi = 3.14159265358979;
```

Tip, hem aralığı hem de işlemleri tanımlar

i Bölüm 6 Bağlantısı

Tip tasarımı kapsamlı bir konudur ve Bölüm 6'da ayrıntılı incelenir. Ancak şunu şimdiden vurgulamalıyız: değişkenin tipi, **kapsam (scope)** ve **ömür (lifetime)** ile birlikte en önemli değişken tasarım konuları arasındadır.

5.3.4 Değer (Value)

Bir değişkenin **değeri**, ilişkilendirildiği bellek hücresinin veya hücrelerinin içeriğidir. Belleği somut (fiziksel) hücreler yerine **soyut (abstract) hücreler** cinsinden düşünmek daha uygun ve doğrudur.

Kavram	Açıklama
Fiziksel bellek hücresi	Çoğu modern bellekte 8 bit (1 byte) — programlar için çok küçük
Soyut bellek hücresi	Değişkenin tipine göre gereken büyüklükte tek birim olarak düşünülür
float (C/Java örnek)	Fiziksel olarak 4 byte kaplar; ancak soyut olarak TEK hücre sayılır
r-value	Değişkenin değeri. Atama ifadesinin sağ tarafında adı geçen değişkenden r-value gereklidir

```
// r-value'ya ulaşım için önce l-value (adres) belirlenmeli:
int a = 5, b = 3, c;

c = a + b;
// Adım 1: a'nın l-value'su (adresi) bulunur → içerik okunur → r-value = 5
// Adım 2: b'nin l-value'su bulunur → içerik okunur → r-value = 3
// Adım 3: 5+3 = 8 hesaplanır
// Adım 4: c'nin l-value'su (adresi) bulunur → 8 yazılır
```

r-value erişimi için l-value önce belirlenmeli

⚠ Dikkat: Kapsam Karmaşıklığı

r-value belirleme her zaman basit değildir. **Kapsam kuralları (scope rules)** bu süreci önemli ölçüde karmaşıktırabilir. Hangi ismin hangi değişkeni referans aldığını belirlemek, scope kurallarının tam anlaşılmasını gerektirir (Section 5.5'te ele alınır).

Σ Bölüm Özeti — 5.1 ile 5.3 Arası Kavram Haritası

Konu	Temel Kavram	Kritik Nokta
5.1 Giriş	Zorunlu diller, von Neumann mimarisinin soyutlamasıdır	Değişken = bellek hücresi soyutlaması; en önemli özellik: tip
5.2.1 Tasarım	İsim tasarımının 2 sorusu: case-sensitivity ve reserved/keyword	Her ikisi de okunabilirlik ve yazılabilirliği etkiler
5.2.2 Biçim	Çoğu dil: harf ile başlar + harf/rakam/_ devamı	camelCase modern standart; PHP/Perl/Ruby özel ön-ekler kullanır
5.2.3 Özel sözcükler	Reserved word yeniden tanımlanamaz; keyword tanımlanabilir	COBOL'un 300 reserved word'ü aşırı bir örnektir
5.3 Sextuple	(isim, adres, değer, tip, ömür, kapsam)	Değişkenin tüm anlamsal boyutları bu altılıyla tanımlanır
5.3.2 Adres	l-value = adres; alias = aynı adrese birden fazla isim	Alias okunabilirlik ve güvenilirliği azaltır
5.3.3 Tip	Değer aralığı + tanımlı işlemler kümesi	Kapsam ve ömür ile birlikte en önemli tasarım konuları
5.3.4 Değer	r-value; soyut bellek hücresi kavramı	r-value'ye ulaşmak için önce l-value belirlenir

RQ Review Questions — Kitap Sonu Soruları (1-5)

RQ-1 What are the design issues for names? / İsimler için tasarım soruları nelerdir?

İsimler için iki temel tasarım sorusu vardır: (1) **Büyük/küçük harf duyarlılığı (Case sensitivity)**: İsimler büyük ve küçük harf ayrımı yapıyor mu? C, C++, Java, C# gibi dillerde evet — rose, ROSE ve Rose birbirinden farklıdır. Bu, okunabilirliği ve yazılabilirliği etkiler. (2) **Reserved word mu, keyword mi?** Dilin özel sözcükleri programcı tarafından yeniden tanımlanabilir mi? Çoğu modern dilde (Java, C, Python) özel sözcükler reserved word'dür — yeniden tanımlanamaz. Fortran gibi dillerde ise keyword'dür, yeniden tanımlanabilir ancak bu karmaşıklığa yol açar.

RQ-2 What is the potential danger of case-sensitive names? / Case-sensitive isimlerin potansiyel tehlikesi nedir?

Case-sensitive isimler iki tehlike yaratır: (1) **Okunabilirlik kaybı**: Görsel olarak benzer isimler tamamen farklı varlıkları temsil edebilir. Rose ve rose aynı gibi görünür ama aralarında hiçbir bağlantı yoktur. Bu, 'benzer görünen yapılar benzer anlam taşımali' tasarım ilkesini ihlal eder. (2) **Yazılabilirlik kaybı**: Programcı doğru büyük/küçük harf kombinasyonunu ezberlemek zorundadır. Java'da Integer.parseInt(...) doğruyken Integer.ParseInt(...) veya Integer.parseint(...) derleme hatasıdır. Bu, gereksiz bilişsel yük demektir. C'de bu sorun, değişken adlarında büyük harf kullanmama *kuralıyla* önlenir. Java ve C#'da ise kaçınılmazdır çünkü predefined isimler zaten karma harf içerir.

RQ-3 What is an alias? / Alias (takma ad) nedir?

Alias: Aynı bellek konumuna erişmek için kullanılan iki veya daha fazla değişken adıdır. Yani birden fazla isim aynı l-value'yu (adresi) paylaşır. Örnek: C++'da `int x = 10; int *p = &x;` yazıldığında, x ve *p alias olur — ikisi de aynı bellek hücrelerini referans eder. Aliasing şu açılardan zararlıdır: • Bir değişkenin değeri, farklı bir isim üzerinden sessizce değiştirilebilir • Okuyucu, iki farklı ismin aynı hücre olduğunu her an akılda tutmak zorundadır • Program doğrulaması (verification) güçleşir Oluşma biçimleri: union tipleri (C/C++), pointer değişkenler, referans değişkenler, alt program parametreleri.

RQ-4 Which category of C++ reference variables always produces aliases?

C++'da **pointer değişkenler bir isimli (named) değişkeni gösterecek biçimde ayarlandığında** her zaman alias üretir. Spesifik olarak: Bir C++ pointer, var olan isimli bir değişkenin adresine (&değişken) atandığında, pointer dereference edildiğinde (*pointer) değişken adıyla alias oluşur. Örnek: `int val = 42; int *ptr = &val; // ptr ve val artık alias`
`*ptr = 99; // val da 99 oldu!` Ayrıca iki pointer aynı konumu gösteriyorsa da alias oluşur; referans değişkenler için de aynı durum geçerlidir.

RQ-5 What is the l-value of a variable? What is the r-value?

l-value (sol-değer): Değişkenin bellek adresidir. Değişken adı bir atama ifadesinin sol tarafında görüldüğünde gerekli olan şeydir. Örneğin `x = 5` ifadesinde `x`'in l-value'su, değer yazılacağı bellek adresidir. **r-value (sağ-değer):** Değişkenin içeriğidir (değeridir). Değişken adı bir atama ifadesinin sağ tarafında görüldüğünde gerekli olan şeydir. Örneğin `y = x + 1` ifadesinde `x`'in r-value'su, bellekteki sayısal içeriktir. **Kritik nokta:** r-value'ye ulaşmak için önce l-value (adres) belirlenmeli, ardından o adresteki içerik okunmalıdır. Bu iki adımlı süreç, kapsam kuralları devreye girdiğinde (hangi isim hangi değişkeni referans ediyor?) ciddi biçimde karmaşıklaşabilir.

PS Problem Set — Alıştırmalar (1-2)

PS-1

Which of the following identifier forms is most readable? Support your decision. `SumOfSales` | `sum_of_sales` | `SUMOFSALES`

En okunabilir form: `sum_of_sales` **Gerekçe:** • `SUMOFSALES` — Tüm büyük harf geleneksel olarak sabitler için ayrılmıştır. Düz metin olarak okunması güçtür; göz, kelime sınırlarını fark edemez. • `SumOfSales` — camelCase yaygın ve kabul görmüş bir standarttır. Okunabilirlik oldukça iyidir; ancak kelime başlangıçları büyük harf farkıyla işaretlendiğinden bazen dikkatli bakış gerektirir. • `sum_of_sales` — Her kelime açıkça görünür ve fiziksel ayraç (alt çizgi) vardır. Okunabilirlik tasarım ilkesi açısından en iyi seçimdir: 'benzer görünen yapılar benzer anlam taşımali' ilkesine en iyi uyan biçimdir. **Not:** Büyük/küçük harf ve alt çizgi kullanımı *programlama stili sorusudur, dil tasarımı sorunu değildir*. Hangi stilin seçileceği ekip/proje standartlarına bağlıdır.

PS-2

Some programming languages are typeless. What are the obvious advantages and disadvantages of having no types in a language? / Tipsiz programlama dillerinin açık avantaj ve dezavantajları nelerdir?

AVANTAJLAR: • **Esneklik:** Değişkenler herhangi türde veri tutabilir; kod daha genel yazılır. • **Kısa ve öz kod:** Tip bildirimleri olmadığından daha az yazım yapılır. • **Hızlı prototipleme:** Tip uyumluluğu endişesi olmadan hızlıca deney yapılabilir. • **Çalışma zamanı esnekliği:** Aynı değişken farklı zamanlarda farklı türler tutabilir (dinamik tip binding ile örtüşür). **DEZAVANTAJLAR:** • **Güvenilirlik kaybı:** Tip uyumsuzlukları derleme zamanında yakalanamaz; hatalar çalışma zamanına taşınır ve tespit edilmesi güçleşir. • **Performans maliyeti:** Tip bilgisi çalışma zamanında takip edilmesi gerektiğinden her değişken için çalışma zamanı tanımlayıcısı tutulmalı ve tip denetimi çalışma zamanında yapılmalıdır. Bu belirgin performans düşüşüne yol açar. • **Okunabilirlik kaybı:** Kod okunurken bir değişkenin hangi türde veri tuttuğu belirsizleşir; anlamayı zorlaştırır. • **Hata yakalamada zayıflık:** Derleyici, mantık hatalarını ve tür uyumsuzluklarını çalışma zamanından önce tespit edemez.

Q Açık Uçlu Quiz Soruları — Sınav Formatı

i Nasıl Çalışmalı?

Aşağıdaki sorular, sınav koşullarında yanıtlanması beklenen açık uçlu sorulardır. Her biri için önce kendiniz yanıt üretmeye çalışın, ardından örnek yanıtı inceleyin.

Q-1

Bir programlama dilinde isim tasarımında case sensitivity (büyük/küçük harf duyarlılığı) kararı alınırken hangi ödünleşimlerin (trade-offs) göz önünde bulundurulması gerekir? Gerçek dil örnekleri vererek açıklayınız.

Case sensitivity kararı şu ödünleşimleri içerir: **1) Okunabilirlik (Readability):** Case-sensitive dillerde Rose ve rose aynı görünür ama farklı varlıklardır. Bu, 'benzer görünen yapılar benzer anlam taşımali' ilkesini ihlal eder ve okuyucu için ek bilişsel yük oluşturur. C++'da bu ciddi bir sorun olabilir. **2) Yazılabilirlik (Writability):** Programcı doğru büyük/küçük kombinasyonunu ezberlemek zorundadır. Java'da parseInt doğru; ParseInt veya parseint yanlış. Bu, derleme hatasına yol açar ve programcıya haksız yük bindirir. **3) Pratik çözümler:** C'de büyük harf kullanmama kuralı bu sorunu büyük ölçüde çözer. Ancak Java ve C#'da pek çok predefined isim zaten karma harf içerdiğinden kaçınılmazdır. **Sonuç:** Case sensitivity esneklik sağlar ancak hem okunabilirliği hem yazılabilirliği olumsuz etkiler. Dil tasarımcısı bu ikisini dengelemeleri gerekir.

Q-2

Değişkeni tanımlayan altılı (sextuple) özelliklerini listeleyin ve her birini kısa bir örnekle açıklayınız. Hangi özellikler birbiriyle ilişkilidir?

Altı temel özellik: **1. İsim (Name):** Değişkeni tanımlayan sembolik etiket. Örn: count **2. Adres (Address / l-value):** Bellekteki konum. Örn: 0x7FFF5ABC. Farklı çağrılarda farklı adres olabilir. **3. Değer (Value / r-value):** Bellekteki içerik. Örn: count = 42 → r-value 42. **4. Tip (Type):** Değer aralığı + işlemler. Örn: int → $[-2^{31}, 2^{31}-1]$ + aritmetik. **5. Ömür (Lifetime):** Belirli bir bellek konumuna bağlı kalınan süre. Stack-dynamic değişkenlerde: fonksiyon çağrısından dönüşe kadar. **6. Kapsam (Scope):** Görünür olduğu deyimler aralığı. **İlişkiler:** • Adres ve Ömür: Değişken bir adrese bağlandığında ömrü başlar, serbest bırakılınca biter. • Tip ve Değer: Tip, değer nasıl yorumlanacağını belirler. • Adres ve Kapsam: Aynı adreste iki değişken olabilir (alias), ancak kapsam farklı olabilir. • Ömür ve Kapsam: Kavramsal olarak farklıdır (Bölüm 5.6) — ömür zamansal, kapsam uzamsal bir kavramdır.

Q-3

Alias (takma ad) kavramını açıklayın. C++'da alias oluşturabilecek üç farklı mekanizmayı örnek kodlarla gösterin. Aliasing neden güvenilirliği azaltır?

Alias tanımı: Aynı bellek konumuna (l-value) erişmek için kullanılan birden fazla değişken adıdır. **C++'da üç alias mekanizması:** 1) Pointer → isimli değişken: `int x=10; int *p=&x; → x ve *p alias` 2) İki pointer aynı konumu gösteriyor: `int *p1=&x; int *p2=&x; → *p1 ve *p2 alias` 3) Union tipi: `union{int i; float f;} u; → u.i ve u.f alias` **Güvenilirlik kaybı nedenleri:** • Bir değişkenin değeri, o değişken açıkça atama almadan farklı bir isim üzerinden sessizce değiştirilebilir → beklenmedik yan etkiler • Okuyucu, tüm alias ilişkilerini aklında tutmak zorundadır → zihinsel yük • Derleyici optimizasyonunu kısıtlar (hangi bellekte değişiklik oldu?) • Formal program doğrulaması (verification) güçleşir

Q-4

Reserved word ile keyword arasındaki farkı açıklayın. Her ikisinin de avantaj ve dezavantajlarını belirtin. Hangi diller hangi yaklaşımı kullanır?

Reserved Word (Ayrılmış Sözcük): Programcı tarafından yeniden tanımlanamaz. Derleme aşamasında özel anlam taşıyan sözcüktür. → Avantaj: Belirsizlik olmaz; derleyici ve okuyucu için açık. → Dezavantaj: Çok fazla reserved word varsa (COBOL: 300 adet!) programcı anlamlı isim bulmakta zorlanır. LENGTH, COUNT gibi doğal isimler kullanılamaz hale gelir. Kullanan diller: Java, C, C++, Python, C# **Keyword (Anahtar Kelime):** Belirli bağlamda özel anlam taşır; başka bağlamlarda yeniden tanımlanabilir. → Avantaj: Programcıya esneklik; isim alanı kısıtlaması yok. → Dezavantaj: Yeniden tanımlama karmaşıklık ve hataya yol açabilir. Fortran'da INTEGER REAL geçerli bir deyimdir — INTEGER değişkenine REAL değeri atanır! Kullanan diller: Fortran **Sonuç:** Reserved word yaklaşımı güvenlik ve okunabilirlik; keyword yaklaşımı esneklik sağlar. Modern diller neredeyse tümüyle reserved word kullanır.

Q-5

I-value ve r-value kavramlarını karşılaştırın. Aşağıdaki C ifadesini analiz edin:
 $a[i] = b[j] + c$; Her alt ifade için I-value mi yoksa r-value mi kullanıldığını ve neden olduğunu açıklayın.

I-value: Bir ifadenin bellekteki konumu (adresi). Atama ifadesinin sol tarafında gerekli. **r-value:** Bir ifadenin değeri (içeriği). Atama ifadesinin sağ tarafında gerekli. **$a[i] = b[j] + c$ analizi:** • $a[i]$ (sol taraf) → **I-value** kullanılır: i indeksindeki dizi elemanının bellek adresi hesaplanır; sonuç oraya yazılacak. • $b[j]$ (sağ taraf) → **r-value** kullanılır: j indeksindeki elemanın adresi bulunur, ardından içeriği (değeri) okunur. • c (sağ taraf) → **r-value** kullanılır: c'nin adresi bulunur, ardından içeriği okunur. • $b[j] + c$ → bu alt ifadenin sonucu geçici bir r-value olur. **Kritik süreç:** r-value'ye erişmek için ÖNCE I-value belirlenmeli, SONRA o adresteki içerik okunmalıdır. Bu süreç kapsam kurallarıyla birlikte ciddi biçimde karmaşıklaşabilir.